SOFTWARE ARCHITECTUUR & TECHNISCH ONTWERP

Bandly — The Artist Operating System

Een modular-monolith blauwdruk voor een Next.js + Spring Boot + PostgreSQL platform dat beginnende artiesten een carrière-backoffice geeft — releaseplanning, positionering, boekingen, rechten, financiën en focus — gebouwd rond één blijvend concept: een geversioneerde *Artist DNA* waar elk gegenereerd document uit put.

BRONMATERIAAL
BANDLY Fase 1 & Fase 2 (NL)

STACK
Next.js · Spring Boot · PostgreSQL

STIJL
Modular monolith, DDD-lite

STATUS
Blauwdruk vóór ontwikkeling

§01

Managementsamenvatting

Bandly is geen marktplaats en geen sociaal netwerk. Het is een single-tenant, AI-ondersteunde **carrière-backoffice** voor een artiest of band met talent maar zonder team — de tool die de rol van manager, publicist, boekhouder en persoonlijk assistent vervult totdat de artiest zich die kan veroorloven. Elk van de twaalf probleemgebieden uit het bronmateriaal (positionering, planning, content, releases, rechten, boekingen, netwerk, financiën, subsidies, communicatie, bestanden, focus) is een facet van dezelfde opdracht: neem de administratieve en creatief-aanpalende last weg, zodat de beperkte vrije tijd van de artiest naar de kunst gaat.

Het architecturale zwaartepunt is de pijplijn **Artist Identity Profile → Artist DNA**: een gestructureerd intakeprofiel dat het systeem vertaalt naar een herbruikbare merkidentiteit (bio's, tone-of-voice, pitchtaal), waar elke andere module vervolgens naar verwijst — zodat een persbericht, een subsidieaanvraag en een Instagram-caption allemaal dezelfde artiest laten klinken, zonder dat de artiest zichzelf telkens opnieuw hoeft uit te leggen.

We adviseren een **modular monolith** — één Spring Boot-deployable, opgedeeld in duidelijk afgebakende, DDD-lite modules, één PostgreSQL-database, één Next.js-frontend — met een interne outbox/event-mechanisme zodat modules op elkaar reageren (een releasedatum zet een checklist op; een checklist-item dat bijna vervalt triggert een reminder) zonder netwerkcalls tussen services. Niets in het bronmateriaal rechtvaardigt microservices, een message broker of een dedicated zoekmachine op dag één; alle drie worden expliciet benoemd als toekomstige opties met een concrete triggervoorwaarde, niet als standaard.

> HOE DIT DOCUMENT TE LEZEN
>
> De secties zijn genummerd omdat ze op elkaar voortbouwen — het domeinmodel in §04 is de input voor elke latere beslissing, en de decision records in §20 zijn precies degene die bepalen of de groeifase in §22 goedkoop of duur wordt. §02 documenteert ook een inhoudelijke correctie op de aanname waarmee deze blauwdruk oorspronkelijk werd aangevraagd; lees die eerst.

§02

Productbegrip

Wat Bandly is, voor wie het is, en een correctie op een cruciale aanname in de oorspronkelijke briefing.

BEVINDING — DE SPLITSING "FASE 1 / FASE 2" BESTAAT NIET IN HET BRONMATERIAAL

Beide PDF's behandelen *dezelfde* twaalf probleemgebieden. "BANDLY Fase 1" (*De doelgroep begrijpen*) beschrijft elk gebied als probleem → hoe AI kan helpen. "BANDLY Fase 2" (*De kansen inschatten*) herhaalt dezelfde twaalf gebieden als input → gegenereerde output, aangevuld met concurrentiepositionering ten opzichte van ChatGPT, Canva, Notion, DistroKid, Spotify for Artists, Mailchimp en spreadsheets. Geen van beide documenten houdt functionaliteit achter voor een latere release. Er is geen gefaseerde *feature*-scope om naar te architecteren.

Deze blauwdruk behandelt daarom alle twaalf gebieden als één MVP-scope (conform jouw richting) en reserveert de termen "Fase 1" en "Fase 2" voor iets architectonisch bruikbaarders: wat als eerste gebouwd wordt, en waar het platform daarna naartoe groeit. Zie §21–§22.

Welk probleem het oplost

Beginnende artiesten — muzikanten, songwriters, producers en de visuele makers om hen heen — hebben meestal een baan naast hun kunst, echt talent, en geen professioneel netwerk om het niet-creatieve werk aan over te dragen. Het bronmateriaal is expliciet over een asymmetrisch gevaar: AI die de *kunst* automatiseert voelt bedreigend, maar AI die de *randzaken* wegneemt voelt als verlichting. Die ene zin is een harde productrandvoorwaarde, geen sfeertekening — zie het risico in §24.

Voor wie het is

Primair — de artiest / bandeigenaar

Zet de Artist DNA op, bezit de bandworkspace, nodigt medewerkers uit, beheert facturatie en instellingen. Expliciet *geen* manager, label of bureau — de eigen positionering van het product is "je career manager voordat je er een kunt betalen."

Secundair — het bandlid / de medewerker

Afgeleid, niet expliciet benoemd als rol: het bronmateriaal noemt taakverdeling "binnen bands of teams" en waarschuwt dat afspraken "verdwijnen in groepsapps." Dit impliceert een gedeelde workspace met meerdere accounts, geen solo-only tool.

OPEN ARCHITECTURALE VRAAG

Er is geen persona gedefinieerd buiten de artiest/band. Er is geen venue-, label-, curator- of fanaccount — die partijen zijn altijd het *doelwit* van iets dat het systeem genereert (een

pitchmail, een perskit), nooit een platformgebruiker. Als een venue- of fangerichte functie ooit gewenst is, is dat nieuwe productscope, geen uitbreiding van wat gedocumenteerd is — benoem dat expliciet in plaats van het via "boekingen" of "netwerk" te laten binnensluipen.

Kernworkflows

1. **Identiteit, eenmalig:** Artist Identity Profile invullen → Artist DNA genereren (bio's, tone-of-voice, pitchtaal, Engelse vertaling).
2. **Een release plannen:** titel/datum/genre/artwork invoeren → een gegenereerde checklist, metadata, pre-save teksten, persbericht, playlist pitch en een evaluatiemoment na release.
3. **Een optreden boeken:** een venue/programmeur-contact vastleggen → een pitch genereren, de reactie bijhouden, herinnerd worden aan follow-up, een rider en setlist maken bij bevestiging.
4. **De boekhouding op orde houden:** split sheets, credits en inkomsten aan een track koppelen; contractsamenvattingen in gewone taal en factuurherinneringen krijgen.
5. **Binnen de week blijven:** een begrensde weekfocus die laat zien wat nu telt en expliciet parkeert wat kan wachten — de enige workflow die als taak heeft om systeemgegenereerde druk te verminderen in plaats van toe te voegen.

Expliciet versus afgeleid

EXPLICIET IN BRON	AFGELEID DOOR DEZE BLAUWDRUK
De 12 probleem-/oplossingsgebieden	Concrete entiteiten, velden en relaties binnen elk gebied
Artist DNA als herbruikbare identiteitsbasis	DNA als *geversioneerde*, doorzoekbare database-entiteit waar andere modules op joinen
Bands/teams verdelen taken	Multi-user org-model met owner/member-rollen
AI stelt concepten op; specialistcontrole nodig bij juridisch/fiscaal	Een concept/goedgekeurd-statuscyclus op alle gegenereerde content
Genoemde concurrerende tools (DistroKid, Spotify for Artists, Mailchimp…)	Deze worden toekomstige integratiedoelen, geen MVP-afhankelijkheden

§03

Requirementsmodel

De twaalf brondomeinen, gegroepeerd naar architecturaal gewicht in plaats van documentvolgorde.

Kern — zonder dit functioneert het platform niet

- **Identiteit & workspace** — accounts, bandlidmaatschap, rollen.
- **Artist DNA** — de identiteitspijplijn waar elke andere module van afhangt.
- **AI-generatiekern** — de gedeelde capaciteit die DNA + modulecontext omzet in concepten.
- **Media & bestanden** — elk domein hangt audio, artwork of documenten ergens aan.
- **Planning & focus** — taken, deadlines en de weekweergave die de deadlines van elke module samenbrengt.

Ondersteunend — waar de productwaarde daadwerkelijk voelbaar is

- Content Studio (social captions, scripts, kalenders)
- Release Manager (releases, tracks, checklists, persmateriaal)
- Rechten & Administratie (split sheets, contracten, facturen, inkomsten)
- Relaties / CRM (boekingspijplijn + netwerkcontacten — zie §06 waarom deze samensmelten tot één module)
- Financiën (begrotingen, break-even, prijsstelling)
- Subsidies & Fondsen (projectplannen, aanvragen)

Toekomst / optioneel — mag de initiële architectuur niet sturen

Synchronisatie met derde partijen (DistroKid, Spotify for Artists API), realtime chat tussen platformgebruikers, native mobiele apps, meerdere bands per gebruiker, marktplaats/ontdekking van andere artiesten of venues, betalingen tussen gebruikers. Elk hiervan krijgt in §22 een bewust uitbreidingspunt, maar wordt niet vanaf dag één meeontworpen.

Feature-uitwerking

FIG. 1 — FEATURE-NAAR-WORKFLOW-MAPPING

DOMEIN	GEBRUIKER	TRIGGER	KERNDATA	OPVALLENDE BUSINESSREC
Positionering	Owner	Onboarding / DNA-vernieuwing	Identiteitsprofiel-antwoorden, referenties, tone-woorden	DNA is geversioneerd; late concepten verwijzen naar een DNA-versie
Planning	Owner, lid	Elke deadline elders ontstaan	Taken, deadlines, weekplan	Weekweergave begrenst zichtbare items — toont nooit alles tegelijk
Content Studio	Owner, lid	Voice memo / tekstinvoer, of release-event	Contentideeën, captions, kalenderslots	Vereist een goedgekeurde DNA vóór eerste generatie
Release Manager	Owner	Nieuwe release opgesteld	Release, tracks, metadata, checklist	Aanmaken van release spawnt automatisch checklist-taken
Rechten & Administratie	Owner	Track afgerond	Split sheet, credits, ISRC, contracten	AI-output krijgt altijd het label "controleer met specialist"
Boekingen (Relaties)	Owner, lid	Nieuw venue-contact	Contact, pitch, rider, interactielog	Elke uitgaande pitch plant zijn eigen follow-up reminder
Netwerk (Relaties)	Owner	Nieuw industriecontact	Contact, notities, introductiemail	Deelt dezelfde Contact-entiteit als boekingen, anders getypeerd
Financiën	Owner	Handmatige invoer of gekoppeld inkomstenrecord	Begroting, uitgave, inkomsten, scenario	Break-even-cijfers herberekenen bij elke gekoppelde wijziging
Subsidies	Owner	Nieuwe aanvraag gestart	Projectplan, begroting, motivatietekst	Hergebruikt DNA + Finance-begroting als generatie-input
Communicatie	Owner, lid	Elke module die uitgaande tekst nodig heeft	Concepten, samenvattingen	Cross-cutting capaciteit van AI Generation, geen eigen aggregate
Bestanden	Owner, lid	Elke upload	Media-asset, structuur per track	Ontbrekende-asset-waarschuwingen verschijn op de releasechecklist

DOMEIN	GEBRUIKER	TRIGGER	KERNDATA	OPVALLENDE BUSINESSREG
Focus	Owner, lid	Wekelijkse cyclus	Prioriteitenlijst, reflectievragen	Genereert nooit nieuwe taken — herschikt/verberg alleen bestaande

§04

Domeinmodel

Concepten afgeleid uit de twee documenten — niet de generieke marktplaatstaal (venue-accounts, aanvragen, matching) die een boekingsplatform zou gebruiken.

FIG. 2 — KERNENTITEITEN

ENTITEIT	DOEL	EIGENDOM VAN	LEVENSCYCLUS
Band	De workspace / tenant-grens — solo-artiest of groep	Owner-gebruiker	Root van vrijwel andere entiteit's tenancy-scope
BandMember	Lidmaatschap + rol van gebruiker in een Band	Band	owner / admin / member-rollen
IdentityProfile	Ruwe onboarding-antwoorden (wie, sound, publiek, referenties)	Band	Vrije tekst; herbewerkbaar, geversioneerd bij opslaan
ArtistDNA	Samengestelde merkidentiteit: bio's, pitch, tone-of-voice, EN-vertaling	IdentityProfile	Gegenereerd, geversioneerd, éé "huidige" versie t
GeneratedAsset	Elk AI-gegenereerd document (caption, persbericht, subsidietekst…)	Band, via bronmodule	concept → bewer goedgekeurd; alti herleiden naar ee DNA-versie
Release	Een plaat/single/EP die uitgebracht wordt	Band	concept → gepla uitgebracht → geëvalueerd
Track	Een nummer binnen een release, of losstaand	Release (optioneel)	Draagt metadata ISRC, credits
MediaAsset	Een opgeslagen bestand: master, stem, artwork, rider, video	Track, Band, of GeneratedAsset	Object storage-p type + metadata
SplitSheet	Songwriter-/producer-verdeling voor een track	Track	Percentages moe optellen tot 100; onveranderlijk na akkoord
Contract / Factuur	Juridische/financiële documenten bij een track of boeking	Track of Contact	AI vat alleen sam stelt nooit binde voorwaarden zelfstandig op
Contact	Een externe partij: venue, programmeur, label, media, curator	Band	Getypeerd via `contact_type`; platformaccount

ENTITEIT	DOEL	EIGENDOM VAN	LEVENSCYCLUS
Interaction	Gelogd contactmoment met een Contact (mail verstuurd, reactie, call)	Contact	Voedt de follow-u reminder-engine
Booking	Een pitch/onderhandeling/bevestigd optreden bij een venue-Contact	Contact	pitched → replie confirmed → pla
Task	Elk actie-item, band- of lid-gescoped	Band, optioneel een bronmodule	Kan systeem-ges zijn (uit Release/Booking of handmatig
Budget / Expense / IncomeRecord	Financiële planning en realisatie	Band, optioneel een Track/Release	Inkomsten kunn een Track gekopp worden voor P&I nummer
GrantApplication	Een subsidieaanvraag in behandeling	Band	Hergebruikt DNA Finance-begrotin generatiecontext
GenerationJob	Een async AI-generatieverzoek + resultaat	Band, bronmodule	queued → runnin done/failed; ruw prompt/response bewaard voor au
Notification	Een in-app/e-mail-nudge afgeleid van een Task- of Interaction-deadline	BandMember	Gerate-limit per dag — zie risico notificatiedruk in

§05

Architectuuroverzicht & Stijl

Advies: modular monolith. Eén Spring Boot-applicatie, één PostgreSQL-database, opgedeeld in de bounded contexts uit §04, die in-process communiceren via duidelijke module-API's en een lichtgewicht intern event-mechanisme (§12) in plaats van HTTP of een broker.

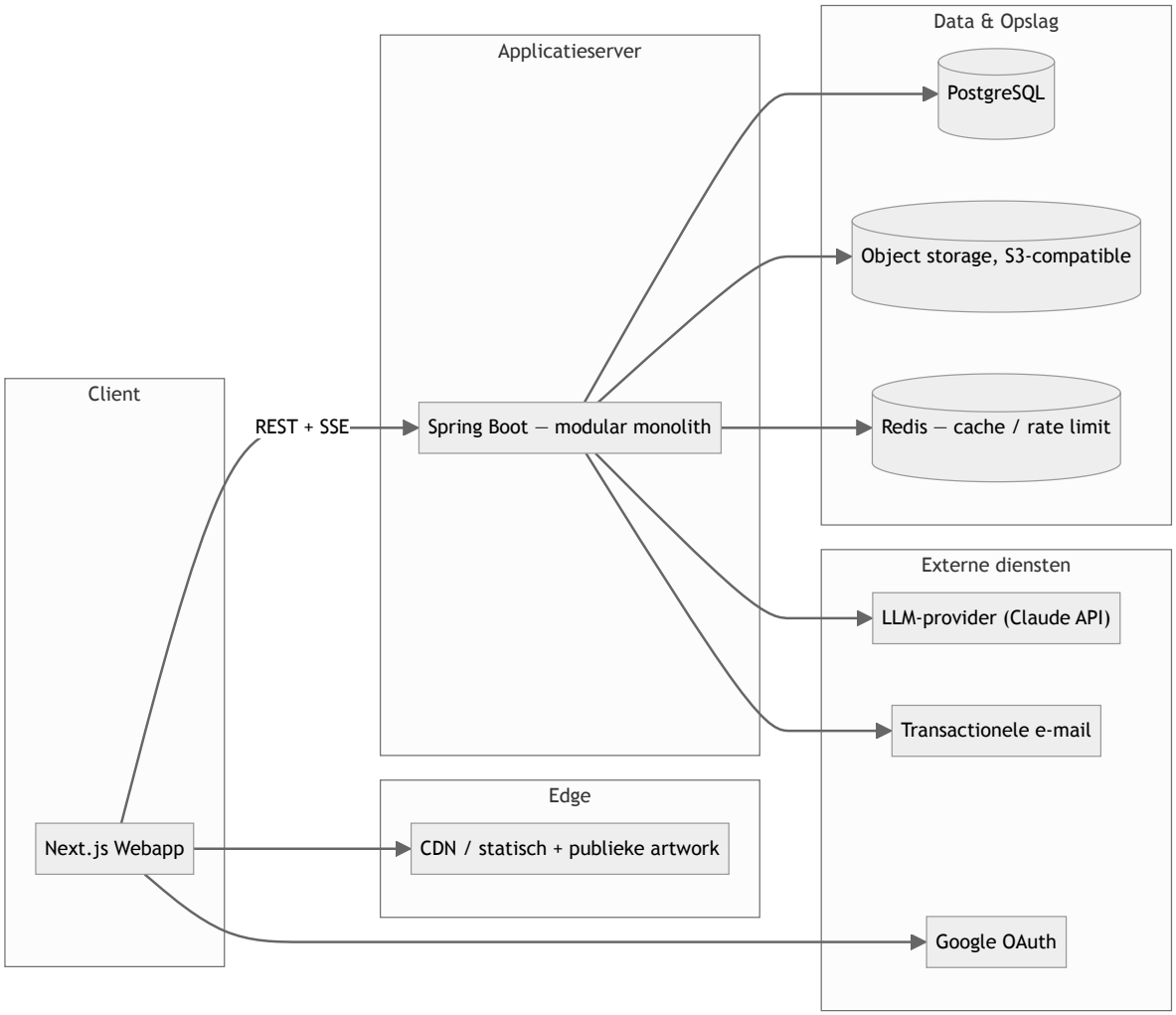
Waarom geen microservices

Bandly's workflows zijn *transactioneel samenhangend*: het aanmaken van een release raakt Release Manager, spawnt Planning-taken, en kan AI Generation en Media triggeren — binnen één gebruikersactie. Dit vanaf dag één opsplitsen in services koopt distributed-transaction-complexiteit (sagas, eventual consistency, service discovery, per-service deploys) voor een team en belastingsprofiel die dat nog niet nodig hebben. Het bronmateriaal beschrijft een single-tenant tool die door één artiest of kleine band tegelijk gebruikt wordt — geen platform dat heterogene, onafhankelijk schalende workloads bedient.

Waarom modulair, en geen platte gelaagde app

Eén platte `service/`-package is precies het faalmodel dat dit vermijdt. Elke bounded context heeft een eigen package, eigen tabellen (op naamgevingsconventie, niet aparte schema's) en biedt de rest van de app een smalle applicatielaag-interface. Die grens maakt latere extractie (§22) een refactor, geen herschrijving.

FIG. 3 — SYSTEEMARCHITECTUUR



§06

Backend-architectuur

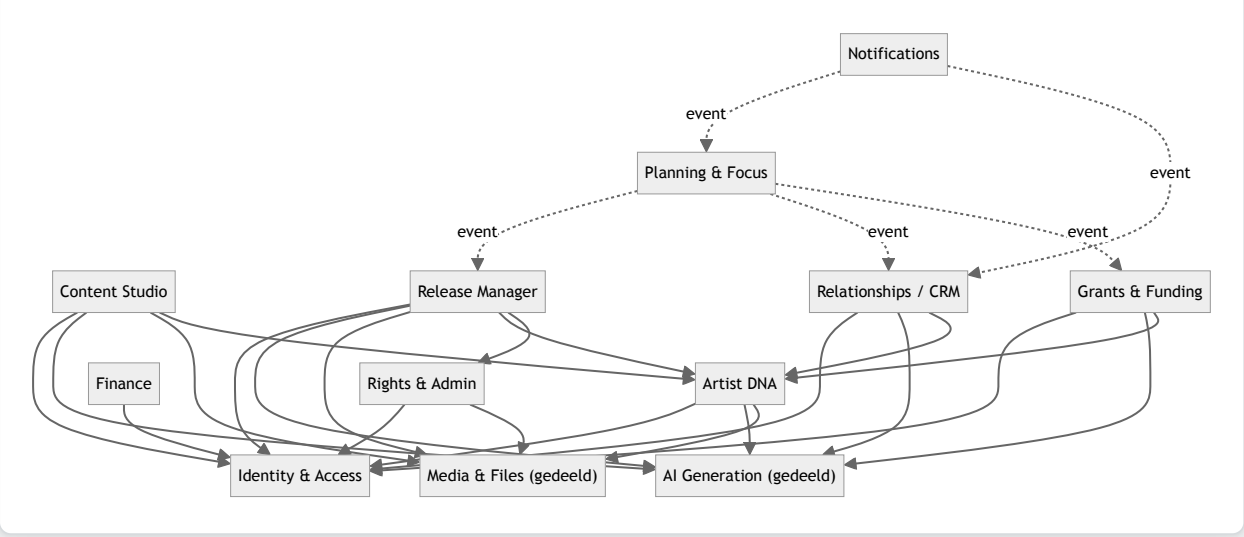
Elke module volgt een lichte hexagonale vorm: een `domain`-package zonder framework-afhankelijkheden, een `application`-package met use cases, en een `infrastructure`-package met Spring Data-repositories en adapters (LLM-client, object storage-client). REST-controllers vormen een dunne `api`-package per module — nooit een god-`controllers/`-map die door de hele app gedeeld wordt.

FIG. 4 — REPOSITORY-INDELING

```
backend/
├── identity/            # gebruikers, bands, lidmaatschap, auth
├── artistdna/            # intake identiteitsprofiel, DNA-generatie + versionering
├── aigeneration/          # gedeelde kern: prompt templates, jobqueue, LLM-adapter
├── media/                 # gedeelde kern: object storage, asset-graaf per track
├── planning/              # taken, weekfocus, deadline-aggregatie
├── content/               # social captions, scripts, contentkalender
├── release/               # releases, tracks, checklists, evaluatie
├── rights/                # split sheets, contracten, facturen, inkomsten
├── relationships/         # contacten, interacties, boekingen (venues + netwerk)
├── finance/                # begrotingen, uitgaven, scenario's, prijsstelling
├── grants/                 # aanvragen, projectplannen
├── notifications/          # in-app + e-mail-levering, reminder-scheduling
├── platform/                # outbox, event bus, correlation ids, gedeelde config
└── BandlyApplication.java
```

Twee modules zijn bewust infrastructureel in plaats van "business"-domeinen: **AI Generation** (prompt-orkestratie + DNA-contextinjectie + async job-afhandeling, gebruikt door DNA, Content Studio, Release Manager, Relationships en Grants) en **Media** (object storage + asset-graaf per track, gebruikt door DNA, Content Studio, Release Manager en Rights). Alle andere modules hangen van deze twee af maar niet van elkaar direct — cross-module-reacties reizen als events (§12), nooit als de ene module die de repository van een andere aanroept.

FIG. 5 — MODULE-AFHANKELIJKHEIDSGRAFIEK



Waarom Boekingen en Netwerk samensmelten tot één Relationships-module

Het bronmateriaal beschrijft vrijwel identieke mechaniek voor "boeken van optredens" (venues, programmeurs) en "netwerk opbouwen" (labels, curators, media): een contact vastleggen, een introductie sturen, opvolgen, de reactie noteren, herinnerd worden. Dit in twee modules opsplitsen zou dezelfde Contact/Interaction/Reminder-machinerie tweemaal dupliceren voor een onderscheid dat in wezen slechts een `contact_type`-waarde is.

Waarom Communicatie geen eigen module is

"Communicatie en professionaliteit" leest als een capaciteit die elk ander domein nodig heeft (een boekingspitch, een subsidiemotivatie, een persmail zijn allemaal gegenereerde tekst), geen domein met eigen entiteiten. Het wordt geïmplementeerd als het outputoppervlak van de AI Generation-module, plus een kleine `Interaction`-log die al bij Relationships hoort voor alles waarvan een blijvend record nodig is van wat wanneer verstuurd is.

§07

Database-architectuur

Eén PostgreSQL-database, tabellen geprefixed per module (`release_` , `rights_` , `rel_` …) in plaats van aparte schema's — houdt cross-module-rapportagequery's eenvoudig terwijl de modulegrens een code-niveau, geen database-niveau, aangelegenheid blijft.

Relationeel versus JSONB

- **Relationeel:** alles wat bevraagd, gefilterd of gejoined wordt — releases, tracks, taken, contacten, boekingen, budgetten, split-sheet-percentages. Dit is verreweg het grootste deel van het schema.
- **JSONB:** twee bewuste uitzonderingen — (1) `identity_profiles.answers` , omdat onboardingvragen kunnen evolueren en de ruwe Q&A als geheel gelezen wordt, niet veld voor veld gefilterd; (2) `generation_jobs.prompt_payload` / `response_payload` , een audit trail van wat naar de LLM gestuurd en teruggekregen is, nooit structureel bevraagd.

REPRESENTATIEVE TABELLEN

FIG. 6 — KERNTABELLEN

TABEL	KERNVELDEN	OPMERKINGEN
bands	id, name, created_at, plan_tier	Tenant-root
band_members	band_id, user_id, role, invited_at	Uniek (band_id, user_id); role enum
artist_dna_versions	id, band_id, identity_profile_id, status, created_at	Eén rij per generatie; `current_version_id`-pointer op band
generated_assets	id, band_id, dna_version_id, source_module, asset_type, content, status	status: draft / edited / approved; index op (band_id, asset_type)
releases	id, band_id, title, release_date, genre, status	status stuurt checklist-taakgeneratie aan
tracks	id, release_id (nullable), isrc, credits_locked	credits_locked bevriest split_sheets
split_sheets	track_id, contributor_name, role, percentage	CHECK: som(percentage) = 100 per track (app-afgedwongen + DB-trigger)
contacts	id, band_id, contact_type, name, org, email	contact_type: venue / promoter / label / media / curator / overig
bookings	id, contact_id, status, proposed_date, gage_estimate	status: pitched / replied / confirmed / played / declined
tasks	id, band_id, assignee_id, source_type, source_id, due_at, done_at	source_type/source_id verwijst terug naar Release, Booking, Grant, of null (handmatig)
budgets / expenses / income_records	band_id, track_id (nullable), amount, category, occurred_at	income_records kunnen optioneel aan een track gekoppeld worden voor P&L per nummer
grant_applications	id, band_id, scheme_name, status, budget_id	status: drafting / submitted / decided
generation_jobs	id, band_id, module, status, prompt_payload (jsonb), response_payload (jsonb)	Async job + auditrecord; zie §12

TABEL	KERNVELDEN	OPMERKINGEN
notifications	id, member_id, task_id (nullable), channel, sent_at, read_at	Gerate-limit bij insert, niet pas bij verzenden

Alle hoofdtabellen dragen `created_at` / `updated_at`; financieel of juridisch relevante tabellen (`split_sheets`, `contracts`, `income_records`) gebruiken soft-delete (`deleted_at`) in plaats van hard delete, omdat deze audit- en, uiteindelijk, fiscaal relevante historie onderbouwen. Al het overige wordt hard verwijderd.

FIG. 7 — ENTITEITSRELATIES



§08

API-architectuur

REST, geversioneerd onder `/api/v1`, band-gescoped in het pad, aangezien vrijwel niets in Bandly buiten de tenancy-grens van een band bestaat. OpenAPI wordt gegenereerd via springdoc; de typed client van de frontend wordt daaruit gegenereerd, niet handmatig onderhouden.

FIG. 8 — REPRESENTATIEVE ENDPOINTS

```
POST   /api/v1/bands/{bandId}/identity-profile
POST   /api/v1/bands/{bandId}/dna/generate
GET    /api/v1/bands/{bandId}/dna/current

POST   /api/v1/bands/{bandId}/releases
GET    /api/v1/bands/{bandId}/releases/{releaseId}
POST   /api/v1/bands/{bandId}/releases/{releaseId}/tracks
GET    /api/v1/bands/{bandId}/releases/{releaseId}/checklist

POST   /api/v1/bands/{bandId}/tracks/{trackId}/split-sheets
POST   /api/v1/bands/{bandId}/tracks/{trackId}/assets       # presigned upload

POST   /api/v1/bands/{bandId}/contacts
POST   /api/v1/bands/{bandId}/contacts/{contactId}/bookings
POST   /api/v1/bands/{bandId}/bookings/{bookingId}/pitch     # triggert AI Generation

GET    /api/v1/bands/{bandId}/tasks?due_before=...
PATCH  /api/v1/tasks/{taskId}

POST   /api/v1/bands/{bandId}/grants
GET    /api/v1/bands/{bandId}/finance/budgets

POST   /api/v1/bands/{bandId}/generation-jobs               # async, SSE-status
GET    /api/v1/bands/{bandId}/generation-jobs/{jobId}/events # text/event-stream
```

- **Paginering:** cursor-based op grootvolume-feeds (taken, interacties, notificaties); offset/limit elders.
- **Idempotentie:** mutatie-endpoints die AI-generatie triggeren accepteren een `Idempotency-Key`-header — LLM-calls kosten geld, en een herhaalde klik mag niet dubbel factureren of dubbel genereren.
- **Fouten:** RFC 7807 `application/problem+json`, consistent over alle modules.
- **Bestandsupload:** client vraagt een presigned URL op bij `/assets`, uploadt rechtstreeks naar object storage, bevestigt daarna — de API-server proxyt nooit audio-/artworkbytes.

- **Rate limiting:** per-band token bucket specifiek op generatie-endpoints (kostenbeheersing, zie §11), los van algemene API-rate limits.

§09

Frontend-architectuur

Next.js App Router, TypeScript, ingedeeld naar dezelfde bounded contexts als de backend, zodat een ontwikkelaar die aan "releases" werkt nooit hoeft te raden welke map het beheert.

FIG. 9 — FRONTEND-STRUCTUUR

```
frontend/
├── app/
│   ├── (auth)/login, register, reset-password
│   └── bands/[bandId]/
│       ├── dna/                 # identiteitsprofiel + DNA-review
│       ├── releases/[releaseId]/
│       ├── content/
│       ├── rights/
│       ├── relationships/       # boekingen + netwerk, tabbed
│       ├── finance/
│       ├── grants/
│       └── focus/                # wekelijks dashboard, standaard startpagina
├── modules/                       # één map per bounded context
│   ├── releases/{api,components,hooks,types}
│   ├── relationships/{...}
│   └── dna/{...}
├── shared/
│   ├── ui/                        # alleen design-system-primitieven
│   ├── api-client/                # gegenereerd uit OpenAPI
│   └── auth/
└── middleware.ts                    # route-level auth guard
```

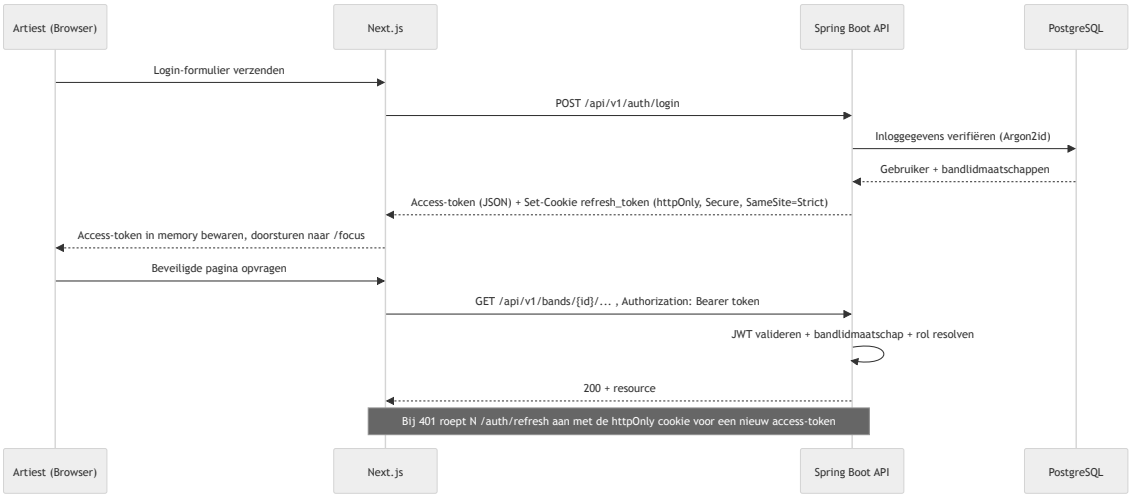
- **Server Components** voor leesintensieve views (releasedetail, contactlijst, budgetdashboard) — fetchen server-side rechtstreeks van de API, geen client-waterval.
- **Client Components** alleen waar interactie het vereist: de DNA-editor, generatie-reviewpanelen (concept accepteren/bewerken/opnieuw genereren), en de SSE-gestuurde generatievoortgangsweergave.
- **Review van gegenereerde content** is een eersteklas UI-patroon, geen chatvenster: elk AI-concept rendert als een bewerkbaar document met een zichtbare status *concept* → *goedgekeurd*, conform het principe "AI stelt op, de artiest beslist" uit §02.
- **State:** serverstate via de gegenereerde API-client + React Query-achtige caching; lokale UI-state (formulierconcepten, focusweergave-filters) via componentstate — geen globale client-store nodig op deze schaal.

§10

Authenticatie & Autorisatie

- E-mail/wachtwoord (Argon2id-hashing) + optioneel Google OAuth.
- Kortlevende JWT access-token (≈15 min), client-side in memory gehouden — nooit in localStorage.
- Refresh-token als httpOnly, Secure, SameSite=Strict cookie; geroteerd bij gebruik.
- E-mailverificatie vereist voordat een band extra leden kan uitnodigen.
- Autorisatie is band-gescoped RBAC: elk verzoek resolvet `(gebruiker, bandId) → rol` via lidmaatschap-lookup; rol bepaalt toegang tot mutatie-endpoints (bijv. alleen owner/admin kan uitnodigen, alleen owner ziet facturatie).

FIG. 10 — INLOGGEN EN GEAUTORISEERD VERZOEK



§11

Beveiligingsarchitectuur

Standaard OWASP-oppervlak

Parameterized queries overal (Spring Data), CSRF beperkt via SameSite-cookies + double-submit token op state-changing formulieren, strikte CORS-allowlist, CSP-headers, dependency-scanning in CI (Dependabot/Snyk).

Generatiekosten als aanvalsoppervlak

LLM-calls zijn factureerbaar; de per-band token-bucket rate limit uit §08 is net zozeer een beveiligingsmaatregel als een kostenmaatregel — zonder die maatregel wordt een gecompromitteerde sessie of gescripte misbruik direct een financiële aderlating.

Uploadveiligheid

Audio-/artwork-/documentuploads worden server-side gevalideerd op type en grootte vóórdat een presigned URL wordt uitgegeven, en gescand (ClamAV of een vendor-scanning-API) voordat ze publiek geserveerd worden.

Rechten- & financiële data

Contracten, facturen en split sheets zijn precies waar het bronmateriaal expliciet om voorzichtigheid vraagt ("juridische of fiscale beslissingen moeten door een specialist gecontroleerd worden"). Deze tabellen zijn audit-gelogd en elke AI-gegenereerde samenvatting draagt een blijvende "geen juridisch/fiscaal advies"-vlag in het datamodel, niet alleen in de UI-tekst.

GEGEVENSBESCHERMING

Een Nederlands/EU-gebruikersbestand impliceert AVG/GDPR standaard, ook al benoemt het bronmateriaal dit niet: recht op export en verwijdering, verwerkersovereenkomst met de LLM-provider, en expliciete gebruikerstoestemming voor elk gebruik van geüploade content (identiteitsantwoorden, voice memo's, songteksten) buiten het genereren van de eigen concepten van de artiest.

OPEN ARCHITECTURALE VRAAG

Er is geen LLM-provider of databeleid gespecificeerd in het bronmateriaal. Deze blauwdruk gaat uit van een provider met een commerciële overeenkomst zonder retentie / zonder training op klantdata (bijv. de Claude API onder Anthropics commerciële voorwaarden) — bevestig dit voordat er gebouwd wordt, aangezien split sheets, contracten en identiteitsprofielen gevoelige content zijn om naar een extern model te sturen.

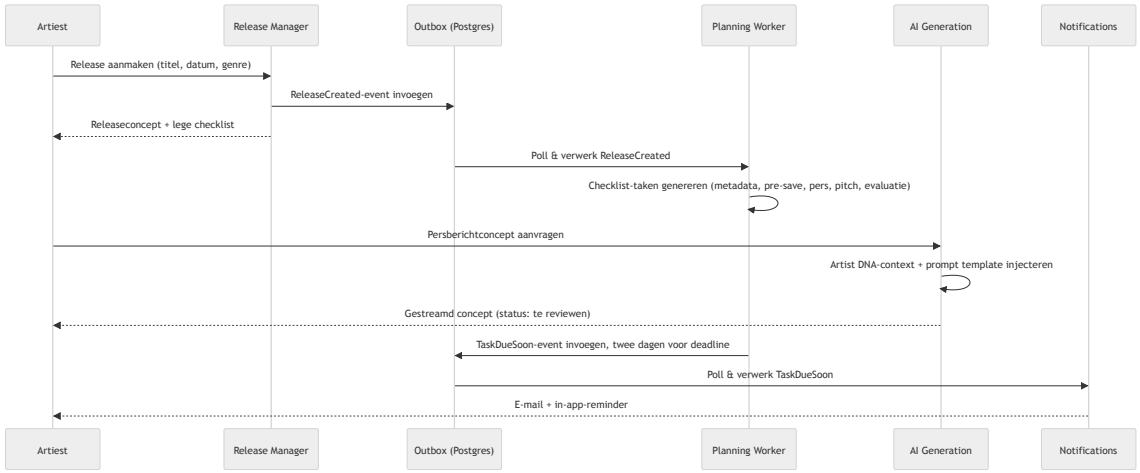
§12

Events & Asynchrone Verwerking

Geen message broker. Een **transactionele outbox-tabel in PostgreSQL**, weggeschreven in dezelfde transactie als de domeinwijziging, uitgelezen door een kleine in-process worker die dispatcht naar abonnerende modules. Dit levert het ontkoppelingsvoordeel (Planning hoeft niet te weten dat Release Manager bestaat om op een releasedatum te reageren) zonder de operationele kosten van Kafka/RabbitMQ — waar niets in het bronmateriaal op deze schaal aanleiding toe geeft.

AI-generatie is van nature asynchroon: een `generation_jobs`-rij beweegt van `queued → running → done/failed`, met voortgang die naar de browser gestreamd wordt via Server-Sent Events in plaats van long-polling of een websocket, omdat het eenrichtings server→client-voortgang is, geen bidirectionele chat.

FIG. 11 — RELEASE AANMAKEN, VAN BEGIN TOT EIND



Migratiepad: als generatievolume of cross-module-fan-out groeit voorbij wat een enkele poller comfortabel aankan, is de outbox-tabel al de durable log die een echte broker zou uitlezen — de poller vervangen door een Debezium/Kafka Connect-pijplijn is een aanvulling, geen herschrijving.

§13

Media- & Bestandsarchitectuur

Directe uitwerking van "bestandsbeheer en workflow" — het bronmateriaal beschrijft zelf een *logische* structuur per track (audio, stems, artwork, lyrics, credits, promo, video, administratie allemaal vanaf één plek vindbaar), wat deze blauwdruk implementeert als datamodel, niet als letterlijke mapstructuur.

- S3-compatible object storage (AWS S3, Cloudflare R2, of gelijkwaardig); elke `MediaAsset`-rij is een pointer, nooit een lokaal schijfpad.
- Presigned upload-/download-URL's — de API-server geeft uit en verifieert, proxyt nooit bytes.
- Publieke assets (goedgekeurde artwork, persfoto's) via een CDN geserveerd; masters, stems en niet-uitgebracht materiaal blijven achter kortlevende signed URL's.
- Automatische naamgevingsconventie (`{band}/{track}/{asset_type}/{version}`) afgedwongen bij upload — directe implementatie van "automatische bestandsnamen voorstellen."
- De releasechecklist kruisverwijst verwachte assettypes per track en signaleert ontbrekende onderdelen — implementeert "waarschuwen wanneer belangrijke bestanden ontbreken" als echt systeemgedrag, geen handmatige controlestap.
- Soft-delete met een bewaartermijn vóór definitieve verwijdering; ondersteunt de AVG-export/verwijderflow uit §11.

§14

Zoeken & Ontdekken

Dit is de sectie waar de mismatch uit §02 het meest telt: omdat Bandly geen artiest-naar-artiest of artiest-naar-venue matching heeft, is er geen "zoek- en matchingengine" om te ontwerpen. Wat het domein daadwerkelijk nodig heeft is in-workspace opzoeken — een contact, een eerdere release, een taak vinden — waar PostgreSQL van nature in voorziet.

- **Fase 1:** Postgres full-text search (`tsvector` + GIN-index) over contacten, releases en gegenereerde assets, gescoped per band. Trigram-indexen voor fuzzy contactnaam-opzoeking.
- **Groeitrigger:** een dedicated zoekmachine (OpenSearch/Elasticsearch) wordt pas gerechtvaardigd als Bandly later echte ontdekking toevoegt — bijv. artiesten die door een gedeelde venue-/curatorgids over bands heen zoeken. Dat is nieuwe productscope (zie §22), geen schaalreactie op de huidige featureset.

§15

Notificaties & Samenwerking

Notificaties

In-app notificatiecentrum + transactionele e-mail (taak vervalt, boeking-follow-up vervalt, DNA-generatie voltooid). Geen pushnotificaties in Fase 1 — er is geen native mobiele app om naar te pushen. Elke reminder is te herleiden naar een `Task` - of `Interaction` -rij; er wordt nooit een notificatie verstuurd die niet correspondeert met een concreet, af te vinken item.

Samenwerking, geen messaging

Het bronmateriaal beschrijft nergens chat tussen platformgebruikers — "communicatie" gaat over uitgaande e-mail/WhatsApp *beter* beheren, en het bandcoördinatie-pijnpunt is specifiek "afspraken verdwijnen in groepsapps." Het juiste antwoord daarop is een gedeelde, toewijsbare takenlijst met lichte comments — geen concurrerend chatproduct. Realtime messaging is expliciet buiten scope; zie §22 als het ooit gewenst is.

§16

Betalingen & Facturatie

OPEN ARCHITECTURALE VRAAG

Geen van beide documenten noemt prijsstelling, abonnementsvormen, of enige betaalflow — tussen platform en artiest, of tussen de artiest en iemand anders. Dit is een echte leemte, geen omissie in deze blauwdruk.

De pragmatische standaard, consistent met "vermijd onnodige complexiteit": Stripe Billing voor platformabonnementen (tiering waarschijnlijk op generatievolume en bandzetels, aangezien AI-kosten de werkelijke marginale kosten van het platform zijn — zie §19), webhook-gestuurde statussynchronisatie naar `bands.plan_tier`, geen kaartgegevens die de eigen database van Bandly raken. Er is geen gebruiker-naar-gebruiker-betaalflow ontworpen, aangezien er geen marktplaatstransactie is om te ondersteunen.

§17

Infrastructuur & Lokale Ontwikkeling

Gecontaineriseerd, cloud-onafhankelijk: Next.js en Spring Boot als twee container images achter een managed load balancer, managed PostgreSQL, managed object storage, managed Redis. Geen Kubernetes nodig op deze schaal — een managed containerplatform (bijv. AWS ECS/Fargate, Fly.io, Render) volstaat en houdt het operationele oppervlak klein.

FIG. 12 — LOKALE DOCKER-COMPOSE-DIENSTEN

```
services:
  web        # Next.js dev server
  api        # Spring Boot, hot-reload
  postgres   # PostgreSQL 16
  minio      # S3-compatible storage-emulator
  mailhog    # vangt uitgaande e-mail op in dev
  redis      # optioneel — rate limiting / cache
```

Omgevingen: **dev** (het compose-bestand hierboven), **staging** (dezelfde container images, geseede/geanonimiseerde data, echte object storage-bucket), **production** (managed Postgres met automatische back-ups + PITR, WAF vóór de API, secrets in een managed secret store — nooit in environment-bestanden die naar de repo gecommit worden).

§18

CI/CD & Testen

Pipeline (GitHub Actions)

1. Lint + typecheck (ESLint/tsc voor frontend, Checkstyle/Spotless voor backend)
2. Unit- en moduletests (backend domeinlogica; frontend componenttests)
3. Integratietests tegen Testcontainers PostgreSQL
4. OpenAPI-contractcheck — gegenereerde client moet overeenkomen met de live spec
5. Database-migratiecheck (Flyway) — dry-run tegen een schoon schema
6. Docker image build + push, getagd op commit SHA
7. Automatische deploy naar staging; production na handmatige goedkeuring

Testpiramide

LAAG	SCOPE	VERPLICHT IN CI
Domein-unittests	Split-sheet-percentageregels, checklist-generatielogica, DNA-versionering	Ja
Repository/integratie	Echte Postgres via Testcontainers	Ja
API-tests	Contract-geverifieerd tegen OpenAPI	Ja
Frontend-component	React Testing Library op generatie-review- en checklist-UI	Ja
End-to-end (Playwright)	Golden paths: onboarding → DNA, release → checklist, boekingspitch → follow-up	Bij merge naar main

§19

Observability & Schaalbaarheid

Observability

- Gestructureerde JSON-logs met een correlation ID die van het Next.js-verzoek door de Spring Boot-call en elke async job die het spawnt loopt.
- Health-/readiness-endpoints voor de eigen checks van het containerplatform.
- Error tracking (Sentry) op zowel frontend als backend.
- **LLM-verbruik als eersteklas metric** — tokengebruik en kosten per band, bijgehouden vanuit `generation_jobs`, zichtbaar op een intern dashboard voordat het ooit aan de artiest getoond wordt. Dit zijn de werkelijke variabele kosten van het platform en de enige metric die echt specifiek is voor dit product.

Schaalbaarheid — dag 1 versus later

Dag 1

Stateless Spring Boot-instanties achter een load balancer, HikariCP connection pooling, CDN voor statische/publieke media, één Postgres-primary. Dit bedient comfortabel duizenden bands.

Bij serieuze schaal

Postgres read replica voor rapportage-/zoekqueries, een aparte workerpool voor generatiejobs (al geïsoleerd achter de outbox), Redis-backed caching voor DNA-/contactopzoeking, horizontale API-schaling. Niet vanaf dag één meeontworpen — wel *toevoegbaar* dankzij de modulegrenzen uit §06.

§20

Architectuurbeslissingen

ADR-01

Modular monolith boven microservices

ALTERNATIEVEN	Microservices per bounded context; een hybride met alleen AI Generation als losse service
REDENERING	Single-tenant workflows zijn transactioneel samenhangend; team en belasting rechtvaardigen geen gedistribueerde complexiteit
CONSEQUENTIES	Eén deploy-eenheid; moduledискipline in code review wordt het echte handhavingsmechanisme
MIGRATIEPAD	Extraheer eerst AI Generation of Media als hun belastingsprofiel (CPU/IO-bound, piekgevoelig) afwijkt van de rest — modulegrenzen maken dit een lichting, geen herontwerp

ADR-02

REST boven GraphQL

ALTERNATIEVEN	GraphQL voor flexibele clientqueries
REDENERING	Band-gescoped resourcevormen zijn voorspelbaar; REST + OpenAPI geeft typed clientgeneratie met veel minder serverside-complexiteit
CONSEQUENTIES	Incidenteel over-/underfetching op dashboardviews, ondervangen met doelgerichte aggregate-endpoints (bijv. `/focus`)
MIGRATIEPAD	Een GraphQL-gateway kan er later voor gezet worden zonder de moduleinterne logica te raken

ADR-03

PostgreSQL als enige database

ALTERNATIEVEN	Postgres + Elasticsearch + Redis + een documentstore, vanaf dag één
REDENERING	Geen requirement in het bronmateriaal vraagt om een tweede datastore; Postgres full-text search en JSONB dekken de flexibele gevallen
CONSEQUENTIES	Eenvoudigere ops, één back-up-/restoreverhaal
MIGRATIEPAD	Redis toevoegen voor cache/rate limit bij contentie; een zoekmachine alleen toevoegen als echte cross-band ontdekking gebouwd wordt

ADR-04

JWT access + httpOnly refresh-cookie

ALTERNATIEVEN	Serverside sessionstore; access-token in localStorage
REDENERING	Stateless access-token houdt de API horizontaal schaalbaar; httpOnly refresh-cookie voorkomt XSS-tokendiefstal

CONSEQUENTIES	Vereist zorgvuldige CSRF-afhandeling op het refresh-endpoint (SameSite=Strict + origin-check)
MIGRATIEPAD	Niet nodig — dit schaalt zoals het is

ADR-05 Next.js App Router, gemengde Server/Client Components

ALTERNATIEVEN	Volledig client-gerenderde SPA
REDENERING	De meeste Bandly-views zijn leesintensieve dashboards; Server Components elimineren client-side fetch-watervallen daarvoor
CONSEQUENTIES	Twee mentale modellen voor datafetching; ondervangen door Client Components te beperken tot écht interactieve oppervlakken
MIGRATIEPAD	n.v.t.

ADR-06 Serverstate via gegenereerde API-client; geen globale client-store

ALTERNATIEVEN	Redux/Zustand globale store
REDENERING	Vrijwel alle state is server-owned en band-gescoped; een cache-on-fetch-patroon dekt dit zonder de overhead van een globale store
CONSEQUENTIES	Heroverwegen als echt cross-cutting client-only state ontstaat (bijv. een realtime samenwerkingslaag in §22)
MIGRATIEPAD	Introduceer een store gescoped op die ene feature, niet app-breed

ADR-07 Postgres outbox in plaats van een message broker

ALTERNATIEVEN	Kafka, RabbitMQ, cloud-native queue
REDENERING	Eventvolume en fan-out zijn klein; de outbox geeft durability en ontkoppeling zonder nieuwe infrastructuur
CONSEQUENTIES	Poll-gebaseerde dispatch heeft latency in de orde van seconden, acceptabel voor reminders en checklist-generatie
MIGRATIEPAD	Outbox-tabel wordt de bron die Debezium/Kafka Connect zou uitlezen

ADR-08 Postgres full-text search boven een dedicated zoekmachine

ALTERNATIEVEN	OpenSearch/Elasticsearch vanaf dag één
REDENERING	Er is geen cross-tenant ontdekking in de huidige scope; in-workspace opzoeken wordt goed bediend door tsvector + trigram

CONSEQUENTIES	Zou echte herarchitectuur vereisen als er later een directory-/ontdekkingsfeature bijkomt — expliciet gemarkeerd in §22
MIGRATIEPAD	OpenSearch naast Postgres introduceren, gevoed vanuit dezelfde outbox-events

ADR–09

S3-compatible object storage vanaf dag één

ALTERNATIEVEN	Lokale/container-schijfopslag voor de MVP
REDENERING	Audiomasters, stems en artwork zijn vanaf de eerste release kernonderdeel van het product; opslag later toevoegen betekent een pijnlijke datamigratie
CONSEQUENTIES	Iets meer setup voor lokale dev (MinIO), volledig gerechtvaardigd doordat een herschrijving vermeden wordt
MIGRATIEPAD	n.v.t. — dit is de juiste keuze vanaf dag één

ADR–10

Geen realtime-messaginglaag in Fase 1

ALTERNATIEVEN	WebSocket-gebaseerde in-app chat tussen bandleden
REDENERING	Niet beschreven in het bronmateriaal; het werkelijke pijnpunt (afspraken die verdwijnen in groepsapps) wordt opgelost door gedeelde taken, niet door chat
CONSEQUENTIES	Geen voor Fase 1-scope
MIGRATIEPAD	WebSocket-gateway is toevoegbare infrastructuur als realtime samenwerking later wordt toegevoegd (§22)

ADR–11

Managed containerplatform boven Kubernetes

ALTERNATIEVEN	Zelfbeheerd Kubernetes-cluster
REDENERING	Twee stateless services en een handvol managed datastores hebben het operationele oppervlak van Kubernetes niet nodig
CONSEQUENTIES	Minder portabiliteit tussen cloudproviders dan met ruwe Kubernetes-manifesten
MIGRATIEPAD	Container images zijn al de deployeenheid — later naar Kubernetes overstappen is een platformwissel, geen herschrijving van de app

§21

MVP-architectuur

MVP-SCOPE alle twaalf brondomeinen, één modular monolith, conform jouw richting in §02.

- **Meegeleverde modules:** Identity, Artist DNA, AI Generation-kern, Media, Planning, Content Studio, Release Manager, Rights & Admin, Relationships (boekingen + netwerk), Finance, Grants, Notifications.
- **Infra:** één Next.js-app, één Spring Boot-app, managed Postgres, managed object storage, Redis voor rate limiting, Stripe voor abonnementsfacturatie.
- **Externe diensten:** één LLM-provider (§11), transactionele e-mailprovider, Google OAuth.
- **Expliciet niet gebouwd:** distributie-integraties, realtime samenwerking, mobiele apps, cross-tenant ontdekking, gebruiker-naar-gebruiker-betalingen.

§22

Groeifase

GROEIFASE Waar de MVP-architectuur naartoe groeit — echt nieuwe capaciteit, geen achtergehouden MVP-scope.

FIG. 13 — VAN MVP NAAR GROEIFASE



WAT IN DE MVP CORRECT MOET ZIJN OM DIT GOEDKOOP TE HOUDEN

1. **Artist DNA als geversioneerde, eersteklas entiteit.** Merkconsistentie achteraf toevoegen aan content die zonder DNA-referentie gegenereerd is, zou een achterstand aan concepten opnieuw moeten genereren betekenen.
2. **Band-gescoped multi-tenancy vanaf dag één.** Multi-user-organisaties achteraf toevoegen aan een single-user-schema is een pijnlijke migratie; band-gescoped beginnen kost nu vrijwel niets extra.
3. **Gegenereerde content als geversioneerde, geauditeerde concepten.** Nodig zodra compliance of juridische review een echte eis wordt — veel goedkoper om vanaf het begin te hebben dan een audit trail achteraf in te bouwen.
4. **Object storage-abstractie, geen lokale schijf.** Al ADR-09 — de dure fout om te vermijden.
5. **Outbox-/eventnaden tussen modules.** Maakt zowel service-extractie als broker-adoptie een aanvulling in plaats van een herbedradingsklus.

§23

Implementatie-roadmap

FIG. 14 — MIJLPALEN

MIJLPAAL	SCOPE	DEFINITION OF DONE
M0 — Fundament	Repo-scaffolding, CI/CD, auth, band aanmaken/uitnodigen	Een gebruiker kan registreren, een band aanmaken, een lid uitnodigen
M1 — Artist DNA	Intake identiteitsprofiel, AI Generation-kern, DNA-generatie + versionering	Een band heeft een goedgekeurde, actuele DNA-versie
M2 — Planning & Notifications	Taken, weekfocus, outbox, reminder-levering	Een handmatig aangemaakte taak stuurt op tijd een reminder
M3 — Release Manager & Media	Releases, tracks, object storage, automatische checklistgeneratie	Een release aanmaken levert een echte, gedateerde checklist op met gemarkeerde assethiaten
M4 — Relationships	Contacten, interacties, boekingen, AI-conceptpitches	Een boekingspitch wordt gegenereerd, verstuurd, en de follow-up wordt op tijd herinnerd
M5 — Rights & Finance	Split sheets, contractsamenvattingen, budgetten, inkomsten	Een track toont credits, rechtenstatus en gekoppelde inkomsten samen
M6 — Grants & Content Studio	Subsidieaanvragen, social content-generatie	Een subsidieaanvraag hergebruikt DNA + een bestaande begroting als generatiecontext
M7 — Productiehardening	Observability, security review, Stripe-facturatie, AVG-export/verwijdering	Klaar voor betalende, niet-pilotgebruikers
M8 — Groeifase	Conform §22, geprioriteerd op echte gebruiksdata	Gedefinieerd zodra M0–M7-gebruiksdata bestaat

§24

Risico's, Open Vragen & Eindadvies

Risico's om nu al tegen te ontwerpen

Het product kan de druk worden die het moet wegnemen

Domein 12 is expliciet: het systeem mag geen extra taken creëren, alleen duidelijkheid over welke bestaande taken ertoe doen. Architectonisch betekent dit dat Planning en Notifications een harde bovengrens nodig hebben op zichtbare items en dagelijks notificatievolume — geen feature om later toe te voegen, een randvoorwaarde voor het datamodel nu.

AI-gegenereerde juridische/financiële content draagt echte aansprakelijkheid

Het bronmateriaal signaleert dit zelf. Elke gegenereerde asset binnen Rights, Finance en Grants draagt een blijvende "specialistcontrole aanbevolen"-vlag in het datamodel, en niets in deze domeinen dient automatisch iets in bij een derde partij.

LLM-kosten zijn de echte marginale kosten van het platform

Anders dan bij de meeste SaaS schalen Bandly's kosten met gebruik op een manier die makkelijk te onderschatten is. De cost-per-band-metric uit §19 en de per-band rate limiting uit §08 bestaan specifiek om dit zichtbaar te houden vóórdat het een crisis wordt.

Scope creep richting "marktplaats"

Boekingen en netwerk lezen dicht tegen een tweezijdige marktplaats aan. Dat zijn ze niet in het bronmateriaal — weersta de verleiding om venue-accounts, aanvragen of matching toe te voegen zonder dat als nieuwe productscope met een eigen requirementstraject te behandelen.

Open vragen vóór de start van ontwikkeling

- Prijsstelling/abonnementsmodel — geen signaal in het bronmateriaal (§16)
- LLM-provider en databeleid (§11)
- Of een gebruiker bij meer dan één band kan horen, en hoe dat geprijsd wordt
- Lokalisatiescope buiten Nederlands/Engels
- Prioriteitsvolgorde voor groeifase-distributie-integraties (DistroKid vs. Spotify for Artists eerst)
- Of bandrollen meer granulariteit nodig hebben dan owner/admin/member

Eindadvies

Als BANDLY morgen in ontwikkeling gaat: één Next.js-app en één Spring Boot modular monolith, ingedeeld in de twaalf modules uit §06, ondersteund door één PostgreSQL-database met het schema uit §07, gedeployed als twee containers op een managed platform met managed

Postgres, object storage en Redis. Artist DNA is de eerste module die na auth gebouwd wordt, omdat de waarde van elke andere module ervan afhangt dat DNA al bestaat. Geen message broker, geen tweede database, geen microservices, geen marktplaats — totdat echte gebruiksdata, niet deze briefing, erom vraagt.